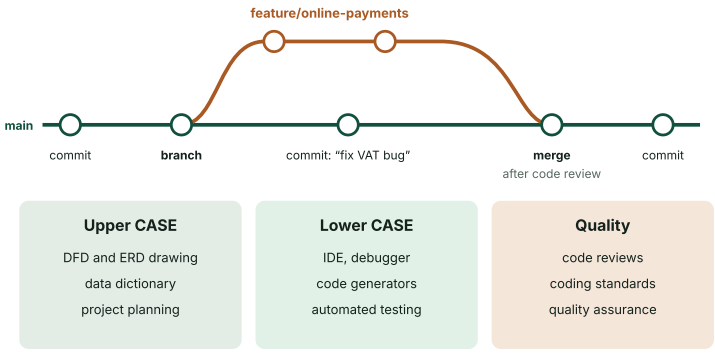
Version management and tools for software projects

AP13 FACT FILE

Specification: students should be able to

- Describe the importance of version management during systems development
- Describe how a version management system records changes and how versions are branched, merged and rolled back, including version management graphs
- Describe the conditions needed for version management to work successfully
- Describe the use of computer-aided software engineering (CASE) tools and collaboration tools
- Describe code reviews, quality assurance and the use of standards in software projects

Key words: version management, version control, repository, commit, revision, branch, merge, conflict, rollback, version management graph, CASE tool, upper CASE, lower CASE, code review, quality assurance, coding standards, collaboration tools



Version control: commits on the main branch, a feature branch that is merged back, and the CASE tools and quality techniques used in software projects

Overview: keeping control of shared code

On any project larger than a single programmer, many people change the same set of files over a long period. Without control, one person's work overwrites another's, nobody can tell who changed what, and a single bad change can break a system with no easy way back. Version management is how a development team keeps track of every change to its code files over time. All files are held in a repository, access is monitored, and each change is recorded with its author, the time it was made and a note explaining it. The team can work on different features in parallel, merge their work together, and return to an earlier working version if an error is discovered. Version management is supported by other tools and practices that help teams produce reliable software: CASE tools that assist with design and coding, collaboration tools that keep distributed teams co-ordinated, code reviews that catch defects early, and quality assurance processes and standards that build quality into every stage of the project.

How version management works

A version management system stores every file in a repository. When a developer finishes a change, they commit it, which saves a new version and records who made the change, when it was made and a message describing what was changed and any knock-on effects that others need to deal with. Each version is numbered, so any earlier version can be identified and retrieved. Developers can create a branch, a separate line of development, so that a new feature can be built without disturbing the main version that others rely on. When the feature is ready, the branch is merged back, combining both sets of changes. If two developers have changed the same lines, the system flags a conflict for a developer to resolve rather than silently losing work. Finally, the team can roll back to any previous version and carry on developing from there. Modern systems such as Git work in this way, and hosted services keep a remote copy of the repository, which also provides a backup.

Version management graphs

A project's history can be drawn as a version management graph. Each node is a version of a file, and an arrow joins each version to the version it was created from. A straight chain shows changes made one after another. When two arrows leave the same node, two developers have worked from the same version in parallel. When two arrows enter one node, those parallel versions have been merged. In the worked example, Ciara and Tomás both start from v2. Ciara's rewrite creates v3, Tomás's allergen labels create v4, and the system merges them into v5. Testing then reveals that v3 introduced a serious error, so v5 is also faulty. The team rolls back to v4, the last stable version, and develops it into v6. On the graph, v6 has an arrow from v4, not from v5, showing that the work from v1, v2 and v4 is kept while the faulty changes from v3 are left out. Drawing these graphs carefully, with every version labelled and every arrow pointing from the older to the newer version, is a common examination task.

Conditions and benefits

Version management only works if the whole team co-operates with it. There must be a consistent method of tracking changes and recording every version number; changes must be reversible; and every team member must be working from the same, up-to-date copy of each file when editing it, otherwise someone may build on out-of-date code and their work will clash with others'. In practice this also means committing regularly with clear messages, so that the history is useful and conflicts stay small. When these conditions are met the benefits are considerable. Several developers can work on the same system at once; branches keep experimental work away from the working version; merging and conflict detection stop changes overwriting each other; a broken change can be undone by rolling back; the full history provides an audit trail showing who changed what and why, which also helps a new team member understand the code; and a remote repository protects the team against losing work if a computer fails.

CASE and collaboration tools

Computer-aided software engineering tools are programs that support or automate parts of the development process. Upper CASE tools help with analysis and design, for example drawing data flow diagrams and entity relationship diagrams, maintaining a data dictionary and planning the project. Lower CASE tools support implementation and testing, such as integrated development environments, code generators, debuggers and automated testing tools. Integrated CASE tools combine both, keeping diagrams, data definitions and code consistent with one another. CASE tools enforce standard notation, produce documentation automatically and can speed up development, but they can be expensive and staff need training to use them well. Collaboration tools help a team, which may be spread across several offices or working remotely, to stay co-ordinated. They include shared repositories with issue trackers for recording bugs and tasks, messaging channels, video meetings, shared documents and wikis, task boards showing who is working on what, and continuous integration servers that automatically build and test the code every time a change is committed.

Code reviews, quality assurance and standards

In a code review, other developers examine code before it is accepted into the main version, often through a pull request on a shared repository. They look for errors, security weaknesses, poor readability and breaches of the team's coding standards. Reviews find defects early, when they are cheapest to fix, and spread knowledge of the code across the team, which reduces the damage if a developer leaves. Reviews can be formal inspection meetings or informal peer checks, and pair programming is a continuous form of review. Quality assurance is broader: it is the set of planned activities, including reviews, walkthroughs, inspections, testing and audits of documentation, that build quality into every stage of a project rather than relying on testing at the end. Standards support quality assurance. Coding standards set rules for naming, layout, commenting and error handling, so that any team member can read and maintain anyone else's code. Process and product standards, such as ISO 9001 for quality management and WCAG for accessibility, give the client confidence that the product has been built properly.

Key terms from the mark schemes

Version management
The process by which a development team keeps track of changes to code files over time, recording who made each change, when and why, and allowing return to earlier versions.
Credited: Version control / source control

Repository
The central store in which a version management system holds all of a project's files and their complete history of changes.

Commit
Saving a change to the repository as a new version, together with a message describing the change.
Credited: Check in

Branch
In version management, a separate line of development used to work on a feature or fix without affecting the main version.

Merge
Combining two parallel versions or branches into a single version that contains both sets of changes.

Rollback
Returning to an earlier, stable version of the code and continuing development from that point.
Credited: Revert

Version management graph
A diagram of a project's history in which nodes are versions and arrows show which version each was created from, showing branches, merges and rollbacks.

CASE tool
Computer-aided software engineering tool: software that supports or automates parts of development, such as diagramming (upper CASE) or code generation and debugging (lower CASE).

Code review
The examination of code by other developers to find errors, security weaknesses and breaches of coding standards before it is accepted.
Credited: Peer review

Quality assurance
Planned and systematic activities, such as reviews, inspections, testing and audits, that build quality into every stage of a project.
Credited: QA

Coding standards
Agreed rules for writing code, such as naming conventions, layout, commenting and error handling, that make code consistent and maintainable.

C C# example

Quality assurance and standards

```
// Coding standard example: meaningful names, a constant, a comment
// Returns the price including VAT, rounded to 2 decimal places
static decimal AddVat(decimal netPrice)
{
    const decimal VatRate = 0.20m;
    return Math.Round(netPrice * (1 + VatRate), 2);
}

Console.WriteLine(AddVat(12.50m));   // 15.00
```

Exam focus

Questions may ask you to explain why version management is important, describe what it records, state the conditions for it to work, or construct a version management graph from a description. Mark schemes credit: tracking every change to code files over time, being able to return to a previous working version, recording who made each change, when and a note describing it, and the three conditions – a consistent method of tracking changes and version numbers, the ability to reverse changes, and everyone working from the same version. Do not describe version management as simply 'saving files' or 'making backups'; link each feature to a team problem it solves, for example branches let a programmer develop a feature without breaking the version others are using. For graph questions, read the description slowly, label every version from v1 upwards, draw parallel versions from the same parent, show merges with two incoming arrows and show a rollback by drawing the new version from the earlier node. For CASE tools, give a specific example; for quality, remember that QA covers the whole process while testing is only one part.

? Check yourself

1. State three items of information a version management system records each time a change is made. [3]
2. Eight programmers at a Belfast software house are working on the same patient-records system at the same time. Explain why the company should use version management. [4]
3. Explain the terms branch and merge in version management. [2]

Answers: 1. Who made the change (1) / when the change was made (1) / a note/message describing the change and any knock-on effects (1). Also accept: the new version number (1). Max 3. 2. Any four from: many programmers work on a shared set of files (1) / without control, changes would overwrite each other / chaos (1) / every change is tracked with who, when and why (1) / the team can return to a previous working version if an error is found (1) / branches allow parallel development without affecting the working version (1) / merging combines work and flags conflicts (1) / provides an audit trail (1) / remote repository acts as a backup (1). 3. Branch: a separate line of development created from a version so that work, e.g. a new feature, does not affect the main version (1) / Merge: combining two lines of development/versions into a single version containing both sets of changes (1).